Доклад

(Слайд 1: Титульный лист)

Уважаемые председатель и члены государственной экзаменационной комиссии! Вашему вниманию предлагается дипломный проект на тему: «Интегрированная система управления потоками пользовательских обращений».

(Слайд 2: Вызовы современных систем поддержки)
В современных условиях эффективное управление коммуникациями с пользователями становится критическим фактором успеха. Однако традиционные системы поддержки сталкиваются с рядом серьезных вызовов.
Во-первых, до 70% всех обращений являются типовыми, но все равно требуют ручной обработки.
Во-вторых, ожидания пользователей растут — они хотят получить ответ в течение 30 минут, в то время как реальное время реакции может достигать нескольких часов.
И, наконец, существующие AI-решения часто работают как «черные ящики», не предоставляя объяснений своим действиям, а риск генерации неточной информации, или «галлюцинаций», недопустим в бизнес-процессах.

(Слайд 3: Цели)
Исходя из этих проблем, были поставлены следующие цели проекта:
Высокая автоматизация: Создать систему, способную самостоятельно разрешать до 60-70% стандартных запросов.

- Обеспечить прозрачность каждого решения через визуализацию пути поиска.
- Разработать гибкое решение, которое можно легко настроить для различных предметных областей.
- Сократить время ответа на типовой запрос с десятков минут до нескольких секунд.

(Слайд 4: Ключевые задачи)
Для достижения этих целей работа была разделена на четыре логических этапа. В основании лежит исследование ограничений текущих систем. Далее — анализ подходов к интеграции с языковыми моделями. На основе этого проектирование модульной и масштабируемой архитектуры. И на вершине — реализация функционального прототипа для проверки гипотез и демонстрации жизнеспособности системы.

(Слайд 5: Существующие решения)
Анализ рынка показал три ключевых подхода к решению проблемы:
Первый — это классические сервис-дески, где вся работа выполняется вручную операторами.

Второй, гибридный, использует AI для маршрутизации запросов к нужным специалистам, оптимизируя поток, но не решая проблему до конца.
И третий, AI-центричный, использует автономных AI-агентов для ведения диалога. Этот подход основан на таких технологиях, как RAG и Graph-RAG, но несет в себе определенные риски.
(Слайд 6: Существующие общие подходы)

(Слайд 7: Анализ аналогов: ключевые выводы)
Проведенный анализ показал, что рынок предлагает фундаментальный компромисс. С одной стороны — скорость и высокая автоматизация, сопряженные с риском «черного ящика» и потерей контроля. С другой — надежность и прозрачность ручной обработки, но ценой высоких издержек и медлительности.
Таким образом, на рынке отсутствует решение, которое эффективно совмещает скорость AI-автоматизации с прозрачностью и надежностью верифицированных знаний.

В ответ на этот вызов была предложена и реализована ключевая идея нашего проекта: использовать большую языковую модель не как генератор контента, а как интеллектуального навигатора.
Вместо того чтобы придумывать ответ, система использует LLM для анализа запроса и прокладывания маршрута по заранее созданной, верифицированной графовой базе знаний. Ответ формируется исключительно из найденного узла, а весь путь к нему предоставляется пользователю. Это частично решает проблему «черного ящика». Далее это будет рассмотрено подробнее.

(Слайд 8: Пользователи системы)
Перед тем как перейти к архитектуре системы, важно понять, для кого создавалась система. Были выделены три ключевые роли:

Конечный пользователь, который хочет получить мгновенный и понятный ответ.
Оператор поддержки, который хочет избавиться от рутины и получать сложные задачи уже с полным контекстом.
И Администратор знаний, которому нужны удобные инструменты для управления логикой системы.
Наша система спроектирована как единая экосистема, повышающая эффективность для каждой из этих ролей.
В разных условиях конечным пользователем может быть как внешний клиент так и, например, внутренний работник.

(Слайд 9: Архитектура системы)
На слайде представлена архитектура системы, спроектированная с учетом принципов гибкости и прозрачности.
Ключевые решения включают:

- Универсальные адаптеры и единый API, что позволяет легко подключать любые каналы взаимодействия — от веб-приложений до мессенджеров и других систем.
- Модульное ядро с разделением логики обработки запросов и команд по принципу CQRS, что повышает производительность и надежность.
- И, конечно, управляемую LLM-навигацию, где внешний AI-сервис используется как “навигатор” для поиска по нашей графовой базе знаний.

(Слайд 10: Итеративный поиск)
Сердцем системы является алгоритм итеративного поиска. Он работает как логическое последовательного итеративного исследования базы знаний.

Система начинает с общих разделов базы знаний.
На каждой итерации LLM анализирует текущий контекст и указывает, в каком направлении «копать глубже».
Система следует этим указаниям, погружаясь в граф знаний.
Цикл повторяется до нахождения точного ответа, а весь пройденный путь сохраняется для полной прозрачности.
(Слайд 11: Итеративный поиск. Пример)
(Слайд 12: Итеративный поиск. Алгоритм)

(Слайд 13: База знаний)
Топливом для нашего алгоритма служит база знаний, построенная на графовой СУБД. Вместо плоского списка статей используется иерархическая структуру, где каждое общее понятие декомпозируется на более частные. Это своеобразная карта знаний, по которой llm-навигатор пытается прокладывать короткий и логичный маршрут к ответу.

(Слайд 14: Интеллектуальные фишки #1)
Рассмотрим два некоторых сценария работы системы.
Первый — автоматическое закрытие. Когда система находит точное решение с высокой уверенностью, она предлагает его пользователю и при подтверждении автоматически закрывает обращение, освобождая операторов от рутины.

(Слайд 15: Интеллектуальные фишки #2)
Второй — умная эскалация. Если задача сложная, система не просто передает ее эксперту, а формирует полный аналитический пакет с историей поиска и гипотезами, экономя оператору время на диагностику.

(Слайд 16: Технологический стек)
Для реализации проекта был выбран современный технологический стек.

Ядро системы написано на TypeScript и работает в среде Deno, что обеспечивает надежность и производительность.

В качестве базы знаний используется лидирующая на данный момент графовая СУБД Neo4j. До этого были попытки создания похожей структуры в MongoDB, но они были не успешны.
Интерфейсы построены на React, а для интеграции с Telegram используется библиотека GrammY.
Интеллект системы обеспечивается интеграцией с передовыми LLM через OpenAI-совместимый API.

(Слайды 17-20: Демонстрация)
А теперь давайте посмотрим, как это работает в реализованном mvp на практике.
(Слайд 17) На первом шаге пользователь задает свой вопрос в простом и интуитивном интерфейсе.
(Слайд 18) Далее система в реальном времени показывает процесс поиска, отображая этапы своего «мышления». Это полностью устраняет эффект «черного ящика».
(Слайд 19) В результате пользователь получает не просто ответ, а точное решение, извлеченное из базы знаний, и, что самое важное, — объяснение пути, по которому этот ответ был найден.
(Слайд 20) Система также предоставляет мощные инструменты для администратора, включая визуальное управление графом знаний.

(Слайд 21: Результаты и Экономический Эффект)
Прототип показал значительный потенциал. Время ответа сокращается с десятков минут до 10-30 секунд, а уровень автоматизации может достигать 60%. Экономический анализ подтвердил целесообразность внедрения: прогнозируемый срок окупаемости проекта составляет 2.1 года, а индекс рентабельности — 1.6.

(Слайд 22: Основные результаты и достижения)
Подводя итог, в рамках дипломного проекта были достигнуты следующие результаты:

- Разработано интеллектуальное ядро, реализующее концепцию «LLM-навигатора».
- Создан работающий прототип с двумя пользовательскими адаптерами, доказывающий гибкость архитектуры.
- И доказана практическая ценность концепции через демонстрацию и экономическое обоснование.

Таким образом, созданный прототип доказывает, что можно эффективно совместить скорость AI-автоматизации с прозрачностью и надежностью верифицированных знаний.

(Слайд 23: Перспективы развития)
Проект имеет значительный потенциал для развития. Дальнейшие шаги включают внедрение механизмов адаптивного обучения и проактивного предложения решений, что позволит трансформировать систему из реактивного помощника в полноценного интеллектуального партнера.

(Слайд 24: Спасибо за внимание!)
На этом мой доклад окончен. Спасибо за внимание! Готов ответить на ваши вопросы.